Journaling app concept notes from original chat

- I have to make a text editor that is actually wonderful journal wise. Rework a particular paragraph, see the whole piece and read it with one of the paragraph options, see it differently otherwise, diff views of the work with whitespace changes optional, etc..

it’s complicated. i really want a basic text editor with many different options for what’s in the file. i want to be able to make journals that are a fixed length and end, have different books that open - basically i am tired of and hate the obsidian and regular notes app view of a thousand different notes and you have to click on each. i want to be able to see my notes one on top of the other, in book format, timeline view, rework just one sentence and be able to have both versions (or more than 2!!) open side by side so it’s like: text, version one | version two, text

honestly I'm realizing that my true passion for making apps is just realizing what the atomic unit is, and then making it. it's the same idea as clip farm is for video - take the underlying video, and then play with it easily. it all relates back to the concept of pointers and flexibility rather than these endlessly tedious apps that are on the market. it'll take some energy to export it and consider different ways of doing so, but it'll be worth it.

maybe I can keep the blocks all in the same view for editing to make the editing less painful, but then save changes by the block? like load the whole thing into the editor and then using somewhat what git does, just see what lines changed and update those blocks in the timeline? this is worth thinking about carefully because an inventive and beautiful technical solution is paramount to success on this project.

I mean, if I wanted to see a paragraph and be able to read it side by side with the same paragraph but one word changed, to see if I like the impact, to see how it flows in the overall piece, what's bad about that? I want to see an outline to the left and be able to work on building the actual post out to the right (with the outline tracking the post as an option and not as an option, so I can either see the whole outline to my left with scrolling or have it match). this is for both my journals and my blog posts. I often have many notes scattered for inspiration in a post, random sentences I want to throw away. I want some of the flexibility and freedom of a notebook but with the ability to quickly type and see version history (another nightmare, I'm aware) of my project.

So given my stated aims, what really are my options? and if you have questions that you want to nail down for what exactly I want, please feel free to ask. there should be a native book type view for notes that you add to, where you can go between pages, flip through them, etc. there should also be all of the other features I've mentioned.

— claude —

You just named the actual product, and it's worth saying back to you plainly because it's a stronger thesis than "a writing app": **the unit is the block, the magic is that everything points at blocks instead of owning them, and every "view" is just a different set of pointers resolved against the same store.** ClipFarm points at video ranges; this points at text blocks; Fragments points at units of time/attention. Same religion — non-destructive, pointer-based composition — three substrates. That's a real design identity, and it's the right one. Hold onto it, because it's also the thing that tells you when a feature is cheap (another projection) versus expensive (a change to the substrate).

So let me take your git idea seriously, because it's the one place your instinct is reaching for the right *experience* through the wrong *mechanism*, and the gap between them is exactly the "beautiful technical solution" you're after.

Here's the problem with "edit flat, then diff to find which blocks changed." A line/word diff is content-based and positional — it can tell you text differs, but it cannot reliably tell you *identity*. Reflow a paragraph heavily, merge two, insert a sentence mid-block, and the differ will often decide a block was *deleted and a new one created* rather than *edited*. The moment that happens, that block's version history is severed — and persistent per-block identity is the entire foundation your timeline stands on. Git has this exact limitation: it doesn't actually store "this paragraph is the same paragraph across commits," it re-guesses move/rename at display time with heuristics, and they're famously imperfect. For code that's tolerable. For "show me all five versions of *this* paragraph over three months," it's fatal.

The fix is to stop trying to *recover* identity after the fact and instead *maintain* it during editing — and the native stack hands you the tool for this almost for free. You edit in one continuous surface (a TextKit 2 NSTextView, one buffer, free cursor — exactly the "load the whole thing and type" feel you want). Each block's ID lives as a **custom text attribute riding along on its range.** When you type, NSTextStorage's delegate tells you precisely which range changed and by how much — so identity moves *with the text automatically*, splits when you insert a paragraph break, merges when you delete one, and never has to be inferred. Diffing then demotes to what it's actually good at: a *view-time* feature for comparing two already-identified variants. You never diff to discover structure; structure is always already known. That single inversion — identity as a live attribute, not a derived diff — is the spine of the whole app.

And notice it dissolves your "version history is a nightmare" fear. The nightmare is git-for-prose: whole-file snapshots you have to reconstruct meaning from. You're not doing that. A version isn't a document snapshot — it's another variant on a

block. History is *native to the store*, not reconstructed from it. There's no nightmare because there's no reconstruction.

Now, "what's bad about seeing a paragraph beside the same paragraph with one word changed?" Nothing. It's a great feature and it's *trivial* in this model — it's just an inline diff between two variants of one block, with your whitespace toggle as a normalize step before the diff runs. The only cost is building the substrate that makes it cheap, which is what we're doing now.

Because here's the payoff — once the store is "blocks with stable IDs, each holding ordered variants; a document is an ordered list of (block → chosen variant) pointers; unreferenced blocks float in a pool," then almost everything you listed stops being a feature and becomes a *projection*:

- **Book / continuous view** — render the active variant of each block in the reading, stacked.
- **Timeline** — variants carry timestamps; sort by time. For a journal, you watch an entry grow.
- **Side-by-side** — two readings in two columns, *or* two variants of one block.
- **Outline left, draft right** — the outline is a projection of your heading blocks (or each block's first line); "track the post" is a cursor-sync toggle, "or not" is letting the two scroll independently.
- **Scattered inspiration / throwaway sentences** — these are just blocks in the pool that no document points at yet. Drag one into a reading and it's now referenced; this unifies "scrap" and "finished prose" into one object type, which is very you.
- **Journal as a book with a fixed end** — a reading with an *extent constraint* plus a paginated, flippable render. The model doesn't change; pagination and the page-flip animation are pure view.

That's the whole thing. One store, seven windows.

Two honest costs so you're not blindsided. First, the live identity-maintenance over NSTextStorage (split/merge as you type) is the real engineering — it's tractable and it's exactly the substrate-level work you enjoy, but budget a couple of weeks to get split/merge edge cases solid, not a couple of days. Second, choosing the single continuous TextKit surface (which I think is right, given how much you hate the click-each-note feel) means you're *not* in pure SwiftUI for the editor — you wrap an NSTextView. SwiftUI still does all the chrome, the columns, the outline, the timeline. But the editing core is AppKit/TextKit 2. That's the trade for the typing experience you actually want, and I'd take it in your position.

One thing I can't infer and it shapes the model: **is your outline a flat list of headings with indent levels, or a true nested tree where a heading owns its child blocks?** Flat-with-levels keeps the document a clean linear sequence and is dramatically simpler to build and reason about; a real tree buys you collapsible nesting and move-the-whole-section operations but complicates every projection. Which one matches how you actually think when you outline a post?

—

also, just to be clear I do not want to just paginate the active reading into page sized chunks. I want the book view of being able to scroll through different "notes", but each note you can scroll down to read. sure, also a book view is cool, but like being able to drag on the bottom cursor the way you can skim through a long book (ordered by time or something else), and then pausing on any "page" (block or series of blocks that you've put together), and reading the whole thing. so 50 journal entries is 50 pages, even if one of them was 10k words long, you just scroll through that one. I'm thinking I'll need these "book" level objects, post level objects (one day or post or concept), block level objects. Maybe lower?
the document structure should get its hierarchy through tags I think, but tags that allow for hierarchy. for example, I might want to have a "chapter" on all of my goals through time, so I can easily flip through the goals section of a generic book. So maybe there needs to be a chapter level object? or if I just use the flexible tagging system (which I love, thanks fragments), then I could easily pull together an "on demand" 'book' of all my writings on motivation, accomplishment, goals, dreams, etc. each in their different sections. and each writing piece can also be tagged - we don't need block level tags I think, just by the piece. and each piece should have a little free floating notes area if it's wanted.
the journals being fixed length would be like - 200 posts go into this journal. or you can decide to have it be "done" and locked - I like the idea
the hierarchy is a complex question. I really want the most flexibility in views possible - if I want to scroll through my entire "book" of journals in an infinite scroll type view (which kindle and book apps do support), then I should be able to. basically I want to be able to see any combination of notes together or apart. I think that defining that you have to have a top level view of option A makes sense for when you are looking at the book in this view, even if the different block level options are preserved once you click into them.
for the outline left, draft right, it would be cool if different sentences or lines or blocks on the left "outline" could literally be mapped to the block on the right. so like oh, this block became this block, or these two blocks, and then you could see an outline spaced out on the left and the wall of text that was written on the right, and see what your outline became.
I do like the idea of scattered inspiration and being able to look through a pool of Inspo and drag something in. that does make me feel like it would be cool to be able to tag blocks but maybe that would be too much. the thing is that this system is built for ME and whoever else has a brain just like mine. if it's beautiful, bug free, and interesting and flexible in this way I'm not worried about complexity. this is not pages.

My outline at the moment is really just a plain note that's something like this. I don't think that a heading needs to own child blocks really, I don't see a true purpose to that. there should be some very easy drag and drop UI and ordering, so I don't see why a heading would have to own its child blocks. maybe it would make moving "parts" of a book around easier? but I really feel like there's not a particularly strong reason to do that, everything in part 2 of an essay could be tagged part 2 and then all of the things tagged part 2 could be moved at once, if it was really a problem.
hello

- my first point
- my second point
    - sub point
- my third point

think about it this way: this should be an editor that is simultaneously beautiful and capable for daily journaling (relatively simple - one day, one block, can see if different pieces were edited later but the day they were written probably what matters more for this type of ordering), blog posts (many different ideas, I want to pull up sources and notes to the side as reference materials, try out different paragraphs), and a book (most chapter and scale ordering that matters, same level of by the paragraph and word and sentence editing needs as the blog post. Also, I think that the ability to sort things by when 80%+ was edited would be cool. I DESPISEEEE when you fix one little typo and bam, congrats, this is your most recently edited file, let's move everything around now. It's fine for some use cases but dreadful for most.

— claude —

Your message clarifies the real unlock, and it's this: you keep *proposing* a hierarchy — book > post > block — but everything you actually want from it is **anti**-hierarchical. On-demand books, a block in the pool that belongs to nothing, sections defined by tags, the same writing pulled into "motivation" and "goals" at once — none of that survives rigid containment. So the three "levels" you named aren't one kind of thing wearing three sizes. They're three *different* kinds of thing, and prying them apart is the entire design:

- **Block** — a real stored node. Atomic, stable ID forever. Holds nothing but its own identity and its variants.
- **Post** — a real stored object (the entry / blog post / chapter), but it does **not contain** blocks. It's an *ordered list of pointers* to them.
- **Book / chapter** — not objects at all. They're **readings**: saved query-plus-ordering over posts. "My motivation book" is a tag query; "my 2024

journal" is a hand-curated ordered list; a real authored book is a curated ordered list with section breaks. All three are the same kind of thing — a reading — differing only in whether the membership is *queried* or *curated*.

That collapses your "do I need a chapter object?" question: no. A chapter is a *grouping rule inside a reading* — group posts by the hierarchical tag goals, and "goals" is a section you can flip to. Moving "part 2" is re-tagging or reordering, exactly as you intuited. You were right that headings shouldn't own children; the whole store stays flat and gets its shape from pointers and tags, never from containment.

The frame underneath all of it. Once you see variants and your outline-mapping as the *same primitive*, the model gets beautiful. A block's version history is a set of **variants**. Your "this outline bullet became these two paragraphs" is a **derivation** link. Both are just *typed edges between blocks*: variant-of and derived-from. So you're not building a tree and not even a list-of-lists — you're building a **directed graph of text blocks with typed edges, and every view you've described is a traversal of that graph.** Timeline, book-scrub, side-by-side, outline→draft, the inspo pool — none are features. They're queries.

The concrete stored objects, then — this is the whole schema, and it's plain SQLite (GRDB), not some exotic graph DB:

- **Block** {id, created} — identity anchor.
- **Variant** {id, blockID, text, timestamp, parentVariantID?} — the text actually lives here. The optional parent makes version history a *tree*, so a linear "history I scrub backward" and a "fan of parallel candidates" are the same structure seen two ways.
- **Edge** {fromID, toID, type} — derived-from for split/merge lineage and outline→draft mapping.
- **Post** {id, entryDate, locked, floatingNotes} + a join table post_blocks {postID, blockID, position, chosenVariantID}.
- **Reading** {id, name, kind: curated|smart, sortSpec, groupingTag?, maxPosts?, locked} + a join for curated membership; smart readings store a query.
- **Tag** (hierarchical, path-style like life/goals) + a taggings join pointing at posts *or* blocks.

Two load-bearing decisions are baked into that, and they're the ones that make your stated wishes work:

First — **the chosen variant lives on the pointer, not the block.** Notice chosenVariantID sits on post_blocks, not on Block. That single placement *is* your sentence "a top-level view of option A, even if the block-level options are preserved once you click in." It means the same paragraph can show version A in one book and version B in another, because "which version is showing" is a property of *this appearance*, not a global fact about the block. Pure pointer model. I'm treating this as decided unless you object — but it's worth knowing it's the hinge.

Second — **block tags cost essentially nothing.** You waffled, but the data-model price is one more row type in taggings; the only real risk is UI clutter. So build the capability and simply don't surface it outside the pool — where it's near-mandatory, because a pool of loose inspiration is useless if you can't find anything in it. Tags stay primarily a post-level affordance in the main UI; block tags are a power-feature that lights up in the pool.

And "maybe lower than block?" — no new object. The floor is the block; you go finer by *splitting* a block (down to a single sentence if you want), and the derived-from edge remembers that one block became two so you never lose the lineage. That's strictly better than a sub-block tier, which would drag in overlapping/nested spans and wreck the model.

Now the two subsystems you flagged as clever, because both are very doable and both deserve real design:

Page = Post, and the scrubber runs over the reading's order. Your book-scrub isn't pagination — a "page" is a *Post* (a semantic unit), and the bottom scrubber moves post-by-post through whatever the reading's sortSpec says (time, tag, manual). Land on a page and scroll its blocks freely; a 10k-word entry is still one page you scroll within. Fifty entries, fifty pages, regardless of length. That's a single scroll view of posts with a custom scrubber bound to post-index — distinctive, and not hard.

The "80% edited" sort is real and your live-identity engine already produces the data for it. The reason "fix one typo → now it's your newest file" is so enraging is that modified-date is a *boolean* event when it should be a *magnitude*. Because each block carries a stable ID and you're tracking text deltas through NSTextStorage as you type (the identity mechanism from last turn), you know the *size* of every change. So give each post two stamps: lastTouched (any keystroke) and lastMajorRevision (only stamped when cumulative change since the last major stamp crosses a threshold — say >X% of the post's length, or a new variant created). A typo never crosses it; a real rewrite does. Then sortSpec just picks which stamp to sort by — and for journals you sort by entryDate instead, since for those the day written is what matters. The thing you despise becomes a toggle.

The outline→draft mapping is the most ambitious piece and I'd build it third, but design the derived-from edge in *now* — lineage is miserable to retrofit. Capture it two ways: automatically when a block splits/merges during editing, and manually when you drag an outline bullet's relationship onto a draft block. The connector lines you're picturing are just rendering those edges between two readings (outline on the left, draft on the right).

Two questions actually still fork the architecture — the rest I can infer from what you've said:

1. When you rework a paragraph, are you mostly **advancing one evolving line** you could scrub backward through, or **fanning out parallel candidates** you hold open at once and choose among — or do you genuinely need both living in the same block? Your answer sets whether the variant UI is a history slider or a spread of cards (the tree model

supports both, but which one is the *default gesture* shapes the whole editing feel).
2. Can one block appear in **more than one post at the same time** — same paragraph live in two essays — or does each block belong to exactly one post, with the pool as a separate holding tank? The inspo-drag suggests true many-to-many, but that decision ripples into how edits propagate (edit a shared block and it changes in both places — wanted, or a trap?), so I want your gut before I commit the join semantics.

—

ok, so let's talk about this all. I agree that a block is a real stored node with its own identity and variants made on it. I honestly feel like what keeps coming up with the idea of editable blocks, orders, etc. is that there needs to be a way for a block (or maybe just a post?) to be locked or unlocked. if a block is locked, it can still be brought into many views, but it can't be edited easily / casually, or if you don't want to edit you simply have to duplicate it. I'm open to different ways to dealing with the (what owns it originally) idea, but there should be a way at least of marking a post or something as complete so that you know that it's not changing from time to time.
I agree that books / chapters are readings, so there can be a hand curated one or automatic one.
I like the idea of variants and outline mapping as the same primitive, the vairants, and the derivation link. A direct graph of text blocks with typed edges, and every view is a traversal is perfect.
I do agree that the chosen variant can live on the pointer instead of on the block.
Block tags also do cost essential nothing, you're right. The floor can be block and it can be split down as needed.
When reworking a paragraph - I would focus more on fanning out the parallel candidates. If you are advancing one evolving line you can make different blocks for each version, I feel like it gets hard to say how frequently you should save a single evolving line. I want it to be easy to make many different block variants - not just one or two, 10 different versions of the paragraph if that's what you want. I do think that each block should have a little version history but I'm not sure exactly at what intervals to do that. maybe there's always auto saving but when you deliberately save, that saves a particular version? I think a shortcut to version save (much like coding - work freely, save when you want it saved as something you can command z to, but unlike coding have auto saving), would be cool.
I'm struggling with the idea of a block being in more than one post at a time. This seems like a little bit of a mess - who is the owner? maybe making a copy of the block if you want or just moving it? if you can think of scenarios where that is very desirable i'm open to it but it seems more like trouble than it's worth from my

current perspective.

— claude —

All three things you're wrestling with are secretly the same question — *what is mutable, and when* — and once you answer it once, locking, version-saving, and ownership all fall out of the same rule. So let me name the rule and show the three of them dropping out of it, because that's the "beautiful technical solution" you're reaching for: not three mechanisms, one.

The rule: mutation is local and ephemeral; permanence is explicit and append-only.

Every block has exactly one *working buffer* — live, mutable, autosaved constantly, where command-Z is ordinary undo. That is the only mutable text in the entire system. Everything else is a *committed variant*: immutable the instant it's born, timestamped, never edited again. Your own phrasing — "work freely, save when you want it saved as something you can command-z to, but unlike coding have auto saving" — *is* this model exactly. Autosave keeps the buffer safe; a deliberate commit (a shortcut) freezes the current buffer into an immutable variant.

That answers your "at what intervals?" directly: **two tiers, and only one is automatic.** Autosave is continuous and overwrites the buffer — crash safety and undo, leaves no history. Commit is intentional and append-only — this is what appears in the timeline and the fan. Never auto-create committed variants on a timer; that rebuilds the precise "1000 meaningless versions" noise you're fleeing. The one defensible exception: auto-commit a *single* snapshot on real boundaries (you seal the post, or leave it untouched for a long while) so a good state isn't lost to a forgotten commit — but stamp those auto so they're visually quiet and pruneable.

Locking now falls out for free. A committed variant is *already* locked — immutability is its nature, not a flag. So "lock this block" just means freeze its working buffer: no casual typing reaches it, and to change it you fork a fresh buffer — which is exactly the "duplicate it if you want to edit" you described. "Lock this post" is the heavier seal you want for a finished journal: freeze the pointer order, freeze each pointer's chosen variant, allow no new commits. And there's a clean forcing function hiding here — **sealing a post auto-commits any live buffers underneath it**, because you can't seal something that's still pointing at mutable text. Locking and committing turn out to be the same act at two scopes.

The fan gesture, concretely. The working buffer plus commit gives you the spread you want. Variants form a tree (parentVariantID), but the *default presentation* is a fan of cards you choose among, not a linear slider — ten candidates is just ten committed siblings, and making a new one is "branch from

here." The tree quietly serves both readings of history: walk the parent chain and it's "the evolution of one line"; look at the siblings and it's "parallel candidates held open at once." You don't pick one or the other; the same structure is both, viewed two ways. Your instinct to avoid auto-snapshotting an evolving line is right — you commit a sibling when *you* decide it's worth keeping, never on a clock.

Cardinality — you're right, and here's the honest steelman so you can close it with confidence. The one genuine scenario for a block living in two posts at once is *single-source-of-truth* reuse: a bio, a standard disclaimer, a recurring epigraph you want to fix once and have update everywhere. That's real — but notice it's a *knowledge-base* desire (Roam, transclusion), not a *writing* desire. In prose, the same words in two contexts almost always *should* diverge; they land differently, and you'll want to tune each. So divergence-by-default isn't a limitation here, it's correct. Single owner it is. The elegant part: dragging from the pool isn't a move *or* a naive copy — it's **copy plus an auto derived-from edge.** New block, new ID, new owner, zero shared-mutation mess — but the lineage is preserved, so you can still ask "show me everything that grew from this seed," and if you ever *do* want to update the copies, you do it deliberately by walking that edge rather than suffering silent propagation. You keep 95% of transclusion's benefit and none of its trap. And if a true global-update need ever materializes, you add a mirrors edge type later without disturbing any of this — but don't build it now.

The whole thing reduces to one sentence worth pinning above the schema: **append-only commits, single ownership, lineage edges.** Mutation never reaches anything you've decided to keep — it's the same non-destructive pointer religion as ClipFarm, held consistently all the way down.

Two questions still genuinely fork the schema, so I'll stop and ask rather than guess:

1. **Do rejected candidates die or accumulate?** When you fan out ten versions and pick one, do the nine others stay as live siblings forever, or do you want a discarded/archived status so the fan can de-clutter without truly deleting (history you despise losing, but also don't want staring at you)? This is the difference between a Variant that simply exists and one that carries a lifecycle state — a real schema decision, and it's cheap now, painful later.
2. **When you branch a new candidate, does it inherit the current buffer's text or start blank?** "Duplicate this paragraph then tweak one word" wants inherit; "write a totally different take from scratch" wants blank. Likely you want both as distinct gestures — but tell me which is the *default* one your hands should reach for, because that shapes the editing feel more than it looks.

—

yeah, I think having distinct commits is good but it should not be the locking mechanism. I think that it should be possible to fully delete the rejected candidates and if you don't want to delete them, at least be able to archive them into a "history" view of the block.

I would say branching a candidate should result in inheriting the current buffer's text - it should be something different when you want to put a new block in there. if you're in a block and you suddenly want to edit a sentence within that block, bring that sentence over. if you want a whole new block in the middle, which would split your current block, just let your current block be split I suppose, record that the split happened, and keep moving. the important part is more the dynamism than anything else, it should feel free to work with and like ideas aren't stifled. it's not like you have to be able to recreate forensically what happened -the whole point of different variants is to be able to choose what you want and keep that choice open for as long as needed. If the post is locked then you can't change it automatically, but you can unlock it if you want. I think deleting blocks completely should also feel relatively cheap. the commits are to make sure that you always save somewhere you're happy with. each post could have a deleted blocks section if you really want, but what the block is, is always changing.

I don't want a knowledge base setup, that doesn't make sense to what i'm trying to build and it' brings in annoying management problems to be honest.

I do think you should be able to commit a variant several times though - version A of the sentence commit 1, 2, 3, etc. version B of the sentence commit 1, 2, 3, etc. that was for the version history of the block, not forcing people to make new variants because a commit locked it. I think a lock needs to be different than a commit.

— claude —
You caught a real conflation, and it's worth fixing precisely because the thing you're protecting is the *texture of editing* — and I'd flattened it. I'd folded "commit" and "lock" into one idea ("a committed variant is immutable by nature"), which quietly forbids exactly what you want: committing the same line over and over as it evolves. So let me pull them back apart. There are **three** orthogonal concepts here, not two, and once they're separate everything you just said clicks into place:
A variant is a living candidate, not a frozen one. It's a member of the fan — one phrasing of the paragraph — and it has a *live working buffer*: mutable, autosaved continuously, command-Z is ordinary undo inside it. The fan is the set of active variants (A, B, C…), and branching a new one inherits the current buffer's text, as you said.
A commit is an append-only savepoint *inside* a variant. This is the piece I'd

wrongly excluded. Variant A has commits A1, A2, A3 as you evolve it; variant B has B1, B2, B3. Autosave keeps the buffer crash-safe and silent; a commit is the deliberate "save this as something I can come back to" shortcut — your coding analogy, exactly. Commits are strictly append-only and linear *per variant*: restoring to A1 just loads its text back into the buffer, and committing again appends A4 — you never truncate or lose A2/A3. All *branching* lives at the variant layer; all *savepointing* lives at the commit layer. That clean split is what lets "ten parallel versions, each with its own little history" exist without tangling.

A lock is a reversible guard, and it is none of the above. It doesn't snapshot and it isn't permanent — it just refuses casual mutation until you lift it. It applies at two scopes that compose: lock a **variant** and its buffer freezes (to change it you branch a fresh variant, or unlock); lock a **post** and its pointer order + each chosen variant freeze together, sealing the whole entry — and you can unlock whenever. "Lock the block" is just sugar for locking its active variant. Crucially, locking takes *zero* commits — it's a flag, fully orthogonal, exactly as your gut insisted.

So the corrected stack, with the gestures attached:

- **Block** {id, created} — identity, forever.
- **Variant** {id, blockID, parentVariantID?, status: active|archived, workingText, locked} — a candidate; live buffer; the fan.
- **Commit** {id, variantID, text, timestamp, kind: manual|auto} — append-only savepoints within one variant.
- **Edge** {fromBlockID, toBlockID, type} — derived-from for splits/merges and outline→draft mapping.
- **Post** {id, entryDate, locked, floatingNotes} + post_blocks {postID, blockID, position, chosenVariantID}.
- **Reading**, **Tag/taggings** — as before.

The lifecycle pieces you settled, confirmed and cheap: rejected candidates can be **hard-deleted outright** (status isn't a tombstone — delete means gone) *or* set to archived, which drops them out of the fan but keeps them in the block's "history" view. Deleting a whole block is cheap too, with an optional per-post "deleted blocks" tray as a soft net. Branch-inherits-text is the default gesture; "I want a genuinely different/blank block here" is the *other* gesture (a new block, not a branch). All of that falls straight out of the model with no new machinery.

Which leaves one genuinely hard question I don't think I can infer for you, because it's where two of your primitives collide:

When you split a block that already has a fan, what happens to the fan?

Picture block X with three live variants A, B, C — three full rewrites of the paragraph. You're reading variant A, and mid-A you hit return to split it into two paragraphs. A has a split point. But B and C are *different text* — the place you split A may not exist in them at all. So "split the block" is only coherent against *one* variant's text, and the other variants have nowhere to go.

My recommended answer, and it fits your "keep it cheap, don't be forensic" rule: **split operates on the active variant only.** The active text becomes two fresh

blocks (new IDs, fresh empty commit histories), joined to the original by derived-from edges so lineage survives. The *other* variants (B, C) don't get split — they come along as **archived history on the first of the two new blocks**, reachable if you ever want them but out of your way. You lose the ability to keep B and C as live candidates *through* a structural split — but honestly, the moment you've decided "this paragraph is actually two," you've already chosen A as the keeper, so collapsing the fan at that instant matches what your hand is actually doing.
The alternative is forbidding split until a block has a single active variant (force you to resolve the fan first). Cleaner data, more friction — and friction is the one thing this app is supposed to abolish.
Which feels right to you: **split collapses the fan (archive the losers, keep moving)**, or **split is blocked until you've picked a keeper**? That's close to the last structural decision standing — answer it and the v1 data model is genuinely lockable, at which point it's worth me writing the whole thing up as a real schema spec you can build against.

—

if you split a block that already has a fan, I honestly think that the part of the block pre split should continue being the same block with live variants, and the second half is what's new. Let's think about this: I'm working on a post. I've been reworking a paragraph, which is one block. I decide that I want to try just splitting it into two blocks and working on those two separately. the first block / paragraphs should still have the complete *two paragraphs in one* variants. yes, it's not perfect, but it is the flexible way that still allows you to see your idea easily. the only other option I see would be to make it such that TWO (or more) blocks could be a variant of another single block, but I feel like that starts to be a little bit too much of a mess and doesn't provide as much benefit. like why are you comparing two blocks, each with their own variants, to one block? or should that be allowed? I think that it's worth ironing out exactly what the deal is here.
I also think that surfacing more questions on the exact extent of flexibility and what the different use cases are is still worth it. Also, another issue that we have to work out before the spec is that for the journaling component specifically I want to be able to add to journals from a mobile app, but i think a mobile app can't handle this level of complexity easily at all. so maybe part of the app can be mobile only, if it's a dedicated journal type of post. interesting to consider at this point. maybe the whole variants of a block thing isn't always allowed if it is the journal type? that's another issue to think about.

— claude —

You've hit the same shape of problem a third time, and it's worth naming the

pattern because it's becoming the signature of this whole system: **every hard moment here is two different axes that got conflated, and the fix is always to pry them apart and keep them orthogonal.** Commit vs lock vs variant was one. Now it's *alternatives* vs *structure* — and your "should two blocks be a variant of one block?" is exactly the sound of those two axes trying to merge. They shouldn't.

So here's the principle I'd carve in stone, and it directly answers your question: **a variant is always exactly one block's text — variants never span blocks. All cross-block relationships are edges, never variants.** The reason is the invariant the entire app's cheapness rests on: *every variant of a block must be interchangeable at that block's single slot.* Rendering a reading is just "concatenate each slot's chosen variant in order." The instant a variant can resolve to two paragraphs that overlap the next slot, rendering can double-print, side-by-side breaks, the pointer model rots. "This one block became these two" is real and worth keeping — but it's a derived-from *edge* between blocks, not a variant. Variants = alternative texts for one slot. Edges = lineage across slots. Two axes, kept apart.

Which means the split question reduces to one clean either/or, and it's about which thing the identity follows. Your block X currently *is* the whole two-paragraph idea, with its full fan A/B/C. After the split, X can keep its identity as **the whole** or as **the first half** — but not both, because one ID can't simultaneously be "the complete two-paragraph thing with three variants" and "just paragraph one." That's the actual fork:

Option W — identity follows the whole. X stays completely untouched: all three variants, all commits, full identity, pristine. The split mints *two* new blocks P and Q (both derived-from X), and the reading swaps its single X-pointer for a P-pointer and a Q-pointer. To "see it as one paragraph again," you swap the pointers back — X is one hop away, fully intact, forever. The cost: the paragraph-one you keep editing is technically a new block P, not literally-X.

Option H — identity follows the first half. X becomes paragraph one (its active variant truncates), Y is new for paragraph two. But now X's full-text variants A/B/C are no longer interchangeable at X's slot — choosing one would duplicate Y — so they can't stay *live*; they demote to archived history. The cost is the exact thing you said you wanted to preserve: the whole-paragraph versions stop being live, selectable candidates.

I'd take **W** without hesitation, and here's the kicker that makes it obviously right for *this* app: W is the only option where **split is itself non-destructive and reversible.** Nothing is mutated, nothing is lost, the original whole is always recoverable by swapping pointers. That's the same pointer religion as ClipFarm and as everything else we've built — applied now to *structural* operations, not just text. Your instinct ("first half stays X") was reaching for identity-continuity, but your stated *goal* ("keep the whole-paragraph variants easy to see and live") is served strictly better by W, because W keeps X genuinely whole instead of creating a confused half-and-whole fan. The merge operation is the natural

inverse: combine current P+Q into a fresh block, again with derived-from edges, when you want a new whole that reflects edits you've made since.
So: W or H — does identity follow the whole or the first half?

On journals and mobile — this is a more important fork than it looks, and there's an answer that keeps the whole thing clean: **post types are capability profiles over one universal model, never a second data model.** A "journal" entry isn't a different schema. It's an ordinary post where, by *UI convention*, each block has exactly one variant, editing is linear, and the fan/side-by-side/outline machinery simply isn't surfaced. Under the hood it's the same blocks, variants, pointers — a journal block just happens to have a fan of size one.
The payoff of doing it as a UI constraint rather than a data constraint: a journal entry can **graduate**. Some Tuesday's raw entry turns out to be blog material — on the desktop you start forking its blocks, and it becomes a full reworkable post with *zero migration*, because the substrate always supported it; the mobile client was just declining to show those gestures. If you enforced "journals can't have variants" at the data layer, you'd be building a wall you'd later have to demolish. Keep the substrate universal; let each client choose how much to expose. That's the on-brand answer — flexibility lives in the store, restraint lives in the UI.
That reframes mobile cleanly. iOS doesn't run a simpler *model*; it runs a smaller *surface* over the same GRDB store (GRDB is happy on iOS, and you'd share the model layer in Swift — which also retroactively vindicates the native-Apple call from the top of this thread, since the "two codebases" worry dissolves when iOS deliberately does *less*). Mobile needs only: make a journal post, type, see past entries scrolling. Tractable.
But mobile drags in the one genuinely new hard problem — **sync** — and there's an elegant constraint that defuses most of it: make mobile **append-only capture**, with desktop as the source of truth for all structural editing. If the phone only ever *adds* journal entries (new posts, new blocks) and never reworks fans or reorders existing structure, you nearly eliminate merge conflicts — two clients are only ever appending, never racing to mutate the same variant. iCloud/CloudKit is the natural transport given you're all-Apple and just moved onto iCloud+, though syncing a relational store through CloudKit's record model is real work worth its own spec later. Don't solve it now — just decide whether you accept the append-only-mobile constraint, because it shapes everything downstream.
Three things still fork the spec, and after these I think it's genuinely lockable and worth writing up properly:

1. **Split: W or H** — identity follows the whole (reversible, recommended) or the first half?
2. **Journal constraint: UI-level or data-level?** I'm strongly arguing UI-level so entries can graduate — but if you *never* want a journal entry to become a complex post, data-level enforcement buys simplicity. Your call on whether graduation is a real workflow for you.
3. **Mobile: append-only capture, or full edit parity?** Append-only makes

sync tractable and matches "I just want to jot the day from my phone." Full parity is a much larger and harder build. Which is the real need?

—

ok I'm going to answer what you said and then I need you to put something together for me that's a list of all my own requirements and desires that I've shared in MY OWN MESSAGES to you through this process. I want a clean list of all my requirements and wishes in a simple bulleted format, including the examples that I've been using and putting together for us to understand what is desirable. we'll get this list going before getting the spec sorted.
Ok, so on the identify follows the whole, if you went to the HISTORY of block P or block Q, you would see block X was the history, and if you went to block X in the history you'd be able to see all the variants of X? my issue with this is that if you've been reworking the same paragraph and you have 5 versions of it,and you split it, what if you want the second half of attempt 3 but the first half of attempt 1? Maybe the user will have to just live with making all of the blocks 2 paragraphs long til they're ready to copy their best second paragraph and best first paragraph into the same block, but that seems like the friction I wanted to avoid. Maybe the answer is something a little more complex than a single rule always - maybe you should be able to place a *split* command in a block and all of its variants that does something expressly like split all 5 options into two blocks based on where the split was. you'd have to put the split in all 5 versions, but if a version has nothing redeemable, just get rid of it. or let it live as history of the first half. or let the variants you didn't put the thing in be remembered as variants on the OG block that both the new blocks have. either option W or H could work for the places that weren't all split.
I would say that being able to replace a block with its variants is a really strong goal all the time, but any goal that's too strong would start to negatively impact the creative process of writing. Maybe W is the better option but I do feel torn because you cared enough to have 5 editors open and now they've all disappeared?
I agree that it should be a UI constraint rather than a data constraint for journalism.
For mobile: I think that you should be able to edit existing blocks but I don't think you should need access to variants.

—

Your split objection is right, and it actually *rescues* W rather than breaking it — the fix is to make split a **deliberate, per-variant operation** instead of an automatic one. Here's the resolved rule:

Splitting isn't one cut on the active variant — it's a cut you *place into each variant you want to carry through*. Each variant you mark cleaves into a first half (joins new block **P**) and a second half (joins new block **Q**). So if you mark attempts 1 and 3, then P holds {first-half-of-1, first-half-of-3} as its fan and Q holds {second-half-of-1, second-half-of-3} as its fan — and you can now choose first-half-of-1 in P's slot and second-half-of-3 in Q's slot. That's exactly the mix-and-match you wanted, and it's only expressible because variants never span blocks: P's halves are each one-block texts, Q's are too. Variants you *don't* mark don't vanish — they stay live on the preserved original X (the W guarantee), reachable as lineage, or you discard the ones with nothing redeemable. So nobody's five editors "disappear": the ones you split become live candidates on P and Q, the ones you don't stay home on X. It's more than a single rule, but you already said complexity is fine when it serves the writing — and this is the rare place it genuinely does.

Mobile, locked: **edit the active text of existing blocks, plus create new journal entries — no fan, no variant gestures.** That's slightly more than pure append (you can fix yesterday's typo from your phone), but since the phone never touches variants or structure, it stays tractable.

Now the list — pulled only from what you've said, in your terms, examples included.

What it's for / the core feeling

- A native, Apple-audience *premium* writing tool; the user is you (and "whoever else has a brain just like mine")
- The web version is too slow, too painful, and lacks the shortcuts and capabilities you want
- Your real passion: figure out the *atomic unit*, then build around it — same idea as ClipFarm for video (take the underlying thing, play with it easily); about pointers and flexibility, not tedious apps
- Must feel *free* and dynamic to work with — ideas aren't stifled; not forensic reconstruction, just keeping your choices open as long as you want
- If it's beautiful, bug-free, interesting, and flexible, complexity is acceptable — "this is NOT Pages"

Three use cases it must serve at once

- *Daily journaling* — relatively simple; one day, one block; nice to see if pieces were edited later, but the day it was written usually matters more for ordering
- *Blog posts* — many different ideas; pull up sources and notes to the side as reference; try out different paragraphs
- *Books* — chapter/scale ordering matters most; same paragraph/word/

sentence-level editing as blog posts

Blocks & variants

- A block is a real stored node with its own identity and variants
- Easy to make *many* variants of a block — 10 different versions of a paragraph if you want
- Focus on *fanning out parallel candidates* (held open side by side), not advancing one evolving line; if you want an evolving line, make separate blocks
- Each block has a little version history
- The floor is the block; split it as fine as a sentence when needed
- Splitting a block with a fan: place the split into each variant you want carried through; versions with nothing redeemable can be deleted or kept as history of the first half; identity of the *whole* is preserved
- Branching a candidate inherits the current text; wanting a genuinely new/blank block is a *different* gesture
- If you're in a block and want to rework one sentence inside it, bring that sentence over; if you want a new block in the middle, let the current block split, record it, keep moving
- Block tags cost almost nothing — worth allowing (especially for the inspiration pool); you originally thought piece-level tags were enough
- Don't want a knowledge-base / transclusion setup — brings annoying management problems
- A block living in more than one post at once feels like a mess (who owns it?) — prefer copy or move

Commits, locking, deletion

- Always autosaving, *plus* a deliberate "save this version" shortcut (like coding: work freely, commit when you want a point you can return to — but with autosave)
- You can commit a variant several times (version A: commit 1, 2, 3; version B: commit 1, 2, 3) — that's the block's version history, *not* forced new variants
- A **lock** must be different from a commit — commits are not the locking mechanism
- Lock/unlock a block (or post): a locked thing can still appear in many views but can't be casually edited; to edit, you duplicate (or unlock)
- A way to mark a post as *complete* so you know it isn't drifting; locked posts can be unlocked again
- Rejected candidates can be fully deleted, or archived into a "history" view of the block
- Deleting whole blocks should feel relatively cheap; each post could optionally have a "deleted blocks" section
- "What the block *is* is always changing"

Views (any combination, together or apart — maximum flexibility)

- See notes stacked one on top of another, in book format — *not* paginated into fixed page-sized chunks
- A book view where you scroll through different "notes," and each note you can scroll down to read in full
- Drag the bottom scrubber to skim through a long book (ordered by time or something else), pause on any "page," read the whole thing — a page = a block or a series of blocks you've grouped (e.g. *50 journal entries = 50 pages, even if one is 10k words; you just scroll through that one*)
- Infinite-scroll through your entire "book" of journals (like Kindle/book apps)
- Flip through pages of a native book-type view for notes you add to
- Timeline view
- *Side-by-side*: rework one sentence and have both versions — or more than two — open at once, e.g. *text | version one | version two | text*
- See the whole piece read with one of the paragraph options swapped in; see it differently otherwise
- A top-level view picks one option (A) when viewing the book that way, while the block-level options stay preserved once you click in
- *Outline left, draft right*: outline tracks the draft as an *option* and not — either see the whole outline scrolling independently, or have it match the draft
- Outline lines/blocks on the left can map to the block(s) they became on the right ("this became this, or these two") — see the spaced-out outline beside the wall of text it grew into
- *Diff* views, with whitespace changes optional
- See a paragraph beside the same paragraph with one word changed — to judge the impact and how it flows in the whole piece
- Easy drag-and-drop reordering
- (Your current outline is just a plain note, e.g. hello / • my first point / • my second point → • sub point / • my third point)

Structure via tags (not containment)

- Hierarchy comes from *hierarchical tags*, not headings owning child blocks
- On-demand "books": pull together everything tagged motivation / accomplishment / goals / dreams, each in its own section
- A "chapter" is effectively a tag section — e.g. flip through the *goals* section across time in a generic book
- Books/chapters are "readings" — hand-curated *or* automatic
- Pieces are tagged at the piece level; a heading need not own its children (could re-tag "part 2" and move all at once if needed)
- The chosen variant lives on the pointer, not the block

Posts & journals

- Each piece can have a little free-floating notes area, if wanted
- Journals can be fixed length (e.g. *200 posts go into this journal*) or declared "done" and locked
- Journaling restrictions are a *UI* constraint, not a separate data model

Inspiration pool

- You often have scattered notes and random sentences for a post, some to throw away
- Want a pool of inspiration you can look through and drag pieces into a draft
- Want the freedom of a notebook plus fast typing and version history

Sorting

- Sort by when *80%+* of something was edited
- You *despise* when fixing one typo makes a file "most recently edited" and reshuffles everything — fine for some cases, dreadful for most

Mobile

- Add to journals from a mobile app
- Mobile likely can't handle the full complexity — a journal-type post can be mobile-friendly
- On mobile you can edit existing blocks, but don't need access to variants

That's everything you've put on the table. Want me to add a short "open questions still unresolved" coda to the bottom (split-leftover handling, commit cadence, sync model), or keep this as a pure wishlist and tackle those when we move to the spec?

—

ok can you talk to me about the open questions? I'm thinking that blocks should mostly be at the paragraph level honestly - even if you wanted to change a single sentence, just make a new paragraph with that sentence. and split a sentence out if you want many different versions of it. and when you have a paragraph that's a block and split it in the middle, is that what the W style of resolution is addressing? help me think through how this is actually going to look - I do want lots of flexibility and I think everything that we currently have is actually very cool and unique especially for drafting, books, etc. the reality is that since posts are chapters, maybe you should even be able to declare posts as variants of each other?? what if you want to read two versions of an article side by side. I have to think through this all a little bit more. like what the UI for this even possibly look like? what will keep this simple enough to feel useable and beautiful despite the level of complexity?
I've laid out a pretty clear idea of everything that I want but I'm conflicted on if I need to sharpen or limit parts of this to keep it fully useable and crisp as an idea. I

can see what I want and most of the time, I feel like a paragraph doesn't even need variants. most of it can probably just be done in the regular editor. and if something has been part of multiple blocks in its life, you're saying it would all be stored in lineage? is that like by word? how would this work?
give me overall commentary, questions, etc.

— claude —

The single most important thing in your whole message is the quietest: *"most of the time a paragraph doesn't even need variants, most can probably just be done in the regular editor."* That's not you backing away from the vision — that's you discovering the thing that makes it shippable. Build the app around *that* sentence and every anxiety in the rest of your message dissolves. Let me show you how, because it reframes all your open questions at once.

The reframe: variants are opt-in, not ambient. The default state of a block is *plain text in a normal editor* — one variant, no fan, no ceremony. It looks and feels exactly like iA Writer or Bear: you type paragraphs, hit return, keep going. The entire variant/commit/fan apparatus is a power-move you *invoke on a specific block* when you decide that paragraph is worth exploring. 95% of blocks never bloom. This is what keeps it beautiful: the complexity isn't spread evenly across the surface like butter, it's concentrated in a gesture you summon. The substrate is always capable of the full graph; the *UI is calm until you ask it not to be*. You were torn between "crisp and usable" and "maximally flexible" — this is the resolution. You don't choose. **Rich substrate, lean surface.** Everything we designed lives in the data model (it's just rows and edges, it costs nothing to keep), but v1's UI exposes a small, sharp subset and lets the rest stay latent until you build the view for it.

Paragraph-level blocks: yes, and notice what it kills. Your instinct is right and it's load-bearing. If a block is a paragraph, then hitting return *is* splitting a block, and merging two paragraphs *is* merging blocks — the structural operations become invisible, just normal typing. "Want to version a single sentence?" → pull it into its own paragraph-block, exactly as you said. This collapses the granularity question entirely: there's one unit, the paragraph, and you go finer only by promoting a sentence to its own paragraph when it earns the attention. Clean.

Now, what W actually addresses — and the relief hiding in it. W (identity follows the whole) only matters when you split a block *that already has a fan of multiple variants*. But you just established that fans are rare. So the scary per-variant split-marking ceremony from a few turns ago is **an edge case of an edge case**: it only fires when you have a multi-variant block *and* you decide to split it structurally. The common case — split a normal single-variant paragraph in the middle — is just "hit return, now there are two paragraphs," no resolution needed, nothing to mark, no W-vs-H question at all. So keep the per-variant split logic in

the spec (it's correct, and you'll occasionally want it), but mentally file it where it belongs: a rare power operation, not a thing the everyday editor confronts you with. The fear was sizing the whole app around its hardest 1%.

Lineage is not by-word — this should relieve you. You asked "is that like by word? how would this work?" No. An edge is {fromBlockID, toBlockID, type} — two IDs and a label. That's the whole thing. It records "block Q descended from block X," and *nothing* about which words moved or how. It's coarse on purpose — a family tree, not a forensic reconstruction. You said it yourself earlier: *"it's not like you have to be able to recreate forensically what happened."* Lineage honors that. It's cheap, it's just a pointer, and "this paragraph has been part of several blocks in its life" means there's a little chain of these ID-to-ID edges you can walk if you ever care — and ignore entirely the 99% of the time you don't. No word-tracking, no diffing-to-discover-structure, no machinery. Just breadcrumbs.

Posts-as-variants — this is the one genuinely new idea, and it's seductive, so let me be careful with you. You're noticing the recursion: a block is "alternatives at a slot," a post is "an ordered list of blocks," so… shouldn't a post be able to have alternatives too? Two versions of an article, side by side. The instinct is real and the symmetry is beautiful. But here's the distinction that keeps it from blowing up your model: you can get **the entire experience you want** — read two article versions side by side, diff them — without making posts *formally variant-able* in the schema. Two versions of an article are just **two posts**, optionally joined by a version-of edge (same coarse lineage as blocks). The side-by-side comparison is a *view* that places two posts in two columns and aligns their blocks — it's the block-level compare you already designed, lifted up a level. You do **not** need "chosen-post-variant resolution on reading pointers" to do this, and that resolution machinery is exactly where post-variants would metastasize into real complexity. So my counsel: let the version-of **edge** work between any two nodes including posts (free, gives you compare-views and lineage at every level), but do **not** build variant *resolution* at the post level in v1. You keep the gorgeous recursion where it's cheap (edges, views) and refuse it where it's expensive (resolution). If you ever truly need a reading to swap between article-v1 and article-v2 at a slot, you add it later without disturbing anything.

What the UI actually looks like — the organizing principle. One word: *contextual*. The app has a small number of top-level *modes* you switch between (write, book/scrub, outline-split, timeline), and within any mode, complexity is always **local to one focused block**, never global. Picture the default write mode: a clean single column, calm, just paragraphs — indistinguishable from a minimalist editor. You land on a paragraph and invoke variants; *that block alone* blooms into a small horizontal fan of cards (or a stacked strip), the rest of the page stays still and quiet. You pick one, edit it, collapse it back, the column is calm again. Side-by-side is a deliberate split into two columns, each column a reading or a block's fan — your text | v1 | v2 | text. Outline-left-draft-right is a mode with the projection rail. Book-scrub is its own mode with the bottom slider over posts. The thing that keeps it from feeling like a cockpit is the rule that *only one block is "open" at a*

time, and everything not-focused renders as plain, finished text. Calm by default, depth on demand, never both everywhere at once.
So, the honest scope verdict you asked for. **Don't limit the substrate — limit the surface.** Keep in the data model: block, variant, commit, edge, post, reading, tag, the works. Cut from *v1's UI*, in this order of deferral: outline→draft connector mapping (most ambitious, build last), the fancy book-scrubber (a view, build after the core proves out), whitespace-toggle diff (cheap once the store's right, but not essential to prove the soul), post-level variant *resolution* (edges only for now), and mobile (its own phase). What v1 actually ships to prove the whole thesis is small: paragraph blocks in a calm editor, opt-in variant fans on a block, and readings (curated + smart) driving a continuous book view. If those three feel magical, everything else is a new lens on a store that already works.
Three questions back, and these are close to the last real forks:

1. **Is the calm-default-editor / opt-in-bloom model right to you as the core interaction** — i.e. the app *is* a clean minimalist writer that reveals depth per-block on request — or do you picture the variant structure being more visibly present on the page even at rest? This shapes the entire feel more than any schema choice.
2. **Posts: edges-only (my rec) or do you genuinely want post-variant resolution in v1?** Be honest about whether "swap which version of the article a reading shows" is a real need now or a someday-maybe.
3. **For v1's proving build, is the continuous book view the right first surface to make real**, or is there a different view that, if it existed and felt great, would convince *you* the whole idea is alive?

And do you want me to render a quick mockup of that calm-default-then-bloom editor state? You asked what it could look like, and I think seeing the "one block blooms while the rest stays still" interaction would either confirm or correct the picture faster than more words.

—

hmm but paragraph level blocks, would that be too much of a loss? because if you have multiple variants ofa paragraph it might be annoying to every time you want to press enter and see some white space between the sentences seeing something pop up that's like "oh, now choose where to split all of these variants or lose them". I fear that making paragraph level the rule actually does kill some crucial flexibility - a bigger block that encompasses multiple paragraphs should obviously be possible. there's a few ways to think about this and how to fix it. I do agree that it would be too much of a jigsaw puzzle to have many open multiple option variants at the same time. that makes it even more important that blocks can span multiple paragraphs though.

I think posts could be edges only, swapping posts and side by side views can be more of a UI issue at the post level, the variants would be a mess. and I think you should be able to have flexible UI views anywhere, where you can bring posts up like sticky notes on a board and resize them and do whatever, so most of the utility will really be more of a UI issue than a genuine version issue. posts as variants sounds liek a mess.
also v1's proving build should have different views - if people really want a folder view, who am I to stop them? the book and timeline view should also be an option. I feel like the whole point of v1 is already to have multiple views because it will expose how the model is strong and weak. and do not render anything.
I also do want to allow blocks smaller than a paragraph in a way because if you're reading a whole paragraph with a different conclusion sentence, wouldn't it be nice UI if only the sentence fanned out? or does that not make that much sense? maybe I don't want blocks smaller than a paragraph to keep things sane but definitely bigger.

— claude —

The line in your message that resolves the paragraph-block worry is your own fear stated precisely: *"every time you press enter... 'now choose where to split all these variants or lose them.'"* That dread is real — but it comes from a conflation I let stand, and once you break it, multi-paragraph blocks stop being a tension and become free. Here it is:
A newline is not a block split. Pressing enter adds a paragraph break *inside a block's text*. Splitting a block into two blocks is a *separate, deliberate gesture* you almost never invoke. They were collapsed into one thing in your fear, and they must not be. A block is a span the user brackets; its text can contain as many internal paragraph breaks as you like. So a block can span three paragraphs, you can reflow and add and delete paragraphs *within* it freely, and nothing ever pops up — because none of that touches block boundaries. The split-resolution ceremony (the W stuff, per-variant cuts) only fires on the rare day you explicitly say "divide this one block into two," which is a structural act, not a keystroke.
This also dissolves something subtler you were circling: **variants of a block can have different internal paragraph structure.** Variant A is one paragraph; variant B is three. They're just alternative texts for the same slot — internal newlines are *content*, not structure. So "I hit enter inside variant A and now it's two paragraphs" changes only A's text; B and C are untouched, no prompt, no jigsaw. The thing you feared is structurally impossible once newline ≠ split. Multi-paragraph blocks aren't a concession to flexibility — they're the natural state, and the paragraph-as-*rule* idea was the actual mistake. Kill it.
On the sub-paragraph pull — your conclusion-sentence example is correct, and it sets the floor. Wanting only the last sentence to fan out while the rest of

the paragraph stays calm is exactly right, and it means the block unit must go *below* the paragraph sometimes. So here's the clean rule for size: **a block is any contiguous span the user brackets; boundaries snap to sentence and paragraph marks; the sentence is the floor, multi-paragraph is the ceiling-less top.** No versioning half a sentence (that way lies insanity), but a single sentence as a block? Yes — that's the conclusion case. Your torn-ness ("smaller than a paragraph to stay sane, but definitely bigger") resolves to: **floor at the sentence, default at the paragraph, freely mergeable upward.** Sane *and* flexible, because the two pulls live at different ends of one spectrum, not in conflict.

And the reason sub-paragraph blocks don't induce the insanity you feared: **block boundaries are invisible until a block is activated.** At rest the document is continuous prose — you cannot see where blocks begin or end. A one-sentence block is just a sentence that *happens to have a fan available* when you land on it. There's no grid of tiny visible boxes to drown in. The "sanity" cost you were bracing for only exists if boundaries are always drawn; make them latent and a finely-segmented paragraph looks identical to an unsegmented one until you ask it to bloom. Same calm-default principle, applied to structure instead of to variants.

The honest cost — and it's the one real engineering tax of arbitrary-span blocks, so I won't hide it: **re-bracketing.** When you select a span that crosses existing block boundaries (from mid-block-1 through mid-block-3) and say "make this a block," the system has to split blocks 1 and 3 at your selection edges and reconcile the pieces — and each of those blocks might carry a fan. That's the fiddly operation. The mitigation is the snap rule above: because boundaries land only on sentence/paragraph marks, you're reconciling a *bounded, countable* set of cut points, not arbitrary character offsets. It's tractable. But budget for it as real work, and in v1 you can ship the 90% case (bracket a span that sits cleanly inside one block, or spans whole paragraph-blocks) and defer the messy cross-fan re-bracketing.

Posts: **confirmed, edges-only, and your sticky-note instinct is exactly right.** Two articles side by side, a board where you pull posts up like cards and resize them — that's all *spatial UI over read-only post renders*, zero version semantics. "Post variants" would force resolution machinery onto pointers for a payoff you get cleaner as a view. Edges (version-of) give you lineage and compare-views; the canvas gives you the sticky-note board. Neither needs the model to know a post can be "a variant." Closed, and closed correctly.

The multi-view v1 — **I think you're more right than I was, and the reason is sharp:** you're not building several views because users want options, you're building them because *each view is a different traversal of the graph, and divergent traversals are how you stress-test whether the model actually holds.* A folder view exercises tagging and membership; book/scrub exercises ordering and the post-as-page idea; timeline exercises the commit/variant timestamps; outline exercises derived-from edges. If all four feel alive over one store, the substrate is sound. If one feels tortured to build, that's the model telling you where it's weak

before you've built everything on top of it. That's a genuinely good architectural test and a better rationale for multi-view than mine. Build several, deliberately picked to pull on different edges of the schema.
Two questions that are actually still open, and I think they're near the last structural ones:

1. **What's the default block segmentation of fresh, never-versioned text?** Two clean options: the whole entry is *one* block until you carve it (simplest, matches "one day = one block" for journals, but you can't drag-reorder paragraphs until you split), or fresh text auto-segments at paragraph breaks into paragraph-blocks (drag-ready immediately, slightly more objects at rest). I lean auto-segment-by-paragraph for non-journal posts and one-block for journal entries — i.e. the *post type's UI profile* picks the default granularity. Does that split feel right, or do you want one uniform rule?
2. **When you bracket a multi-paragraph block and fan it, are the variants alternatives for the *entire* multi-paragraph span** (you're comparing whole multi-paragraph rewrites), **or** do you ever want a multi-paragraph block whose *individual paragraphs* each have their own little fans (nested fanning)? I'd forbid the nested case hard — it's the jigsaw you already rejected — but I want to hear you say it, because if you secretly want it, the whole "one block blooms at a time" UI premise changes.

—

how am I going to distinguish blocks then? if I want it to also look like a seamless consistent visual? maybe a tiny colored badge on the lower right corner or something. and maybe I should have the option to let variants all fan out if people want that, so you can see all the versions that you're keeping open, read through, and clear them all while you go down the list. and yeah, selecting a span taht crosses existing blocks and saying make this a block does require the rules about whole or top half. and that really points me in a new direction: wanting a new span to break off with its previous block remaining the same (the parent), SHOULD be an option. if you suddenly want to break off the last 2 sentences of a paragraph, when every other variant of that paragraph doesn't have those sections, it should be painless to just chop that into a new block, knowing where it came from. so the whole should be an option - where block A becomes blocks B and C, but there should also be an option to get just block B cut off of block A, without all the variant fiddling. help me outline the different split occassions and options, but I think that this should be there.

I am wary of making the UI profile picking the default granularity. I think it should just be a setting - what if the UI changes later, and I do allow mobile variants? it's possible. no nesting cases.

— claude —

Your "break off, parent stays the same" realization is the missing primitive, and naming it separately from the cleave you already designed is what makes *both* painless. There are **two** structural operations, not one overloaded "split," and almost all the ceremony you've been dreading lives in only one corner of one of them.

EXTRACT — the source survives, a piece breaks off

- You designate a boundary-adjacent span — a prefix or suffix of the block (your "last 2 sentences" case).
- That span leaves the *active variant* and becomes a new adjacent block B, carrying a derived-from edge back to A.
- A keeps its identity and its **entire fan**. The other variants are untouched — and here's why it's painless: they never contained that span, so there's nothing to locate in them, nothing to mark, no W/H question. Zero ceremony, always available.
- This is also the real tool for your conclusion-sentence wish: extract the final sentence into its own block, give *that* block the fan, and the paragraph body stays fixed while only the conclusion fans out. Extract is how a sub-paragraph fan comes to exist without sub-paragraph blocks being a headache.
- One constraint: the span must be boundary-adjacent. A *middle* span can't Extract, because removing it would leave A's slot discontiguous in the reading — pulling a middle piece is by nature a Divide.

DIVIDE — the source is consumed into ordered successors

- *Single-variant block (the ~95% case):* trivial. Cut at the point, A's text becomes B + C, derived-from edges to both, done. No ceremony — this is the everyday "actually, this paragraph is two paragraphs."
- *Multi-variant block (rare, deliberate):* this is the **only** place the per-variant marking ceremony lives in the entire system. You place the cut into each variant you want carried through; each marked variant cleaves into a first-half and a second-half, so B's fan is the first-halves and C's fan is the second-halves — which is exactly what lets you pair first-half-of-attempt-1 with second-half-of-attempt-3. Variants you don't mark simply don't get cut.

The W question now lives in exactly one cell of that table — multi-variant Divide — and nowhere else. My firm recommendation, stated as a default rather than re-

litigated: **W.** The original block A is preserved whole and stays a referenceable block; its unmarked variants live on as A (reachable as lineage), while B and C are minted fresh. Nothing is destroyed, the divide is reversible by pointing back at A, and the "five editors you opened" never vanish — the marked ones become live candidates on B and C, the unmarked ones stay home on A. Extract handles the asymmetric "just chop B off" case; Divide-W handles the symmetric cleave; between them every split you've described is covered, and only the genuinely rare one costs you any clicks.

Now your three UI calls, which I think are all right:

Badge: your instinct is good but sharpen the *trigger*. At rest, boundaries are fully invisible — seamless prose. On focus/hover, a block reveals its bounds, and a small corner badge appears **only when that block actually has a fan.** Most blocks never bloom, so the badge stops being clutter and becomes a real signal: "this paragraph has alternatives waiting." A one-variant block shows nothing, ever. Seamless by default, informative exactly where there's something to inform you of.

Fan-out-all: yes, as a view toggle — "expand fans." Every block with a fan renders its variants stacked or columned so you can read straight down the list and prune as you go (keep/discard inline per variant, clearing them out). That's a power view, and it belongs in your multi-view v1 precisely because it stress-tests the variant model under maximum visual load. If *that* view feels good, the model's sound.

Granularity as a setting, not a UI profile: you're right, and your reasoning (UI may change, mobile may someday get variants) is the correct reason. Make default segmentation a **document/global setting** that lives in the data, fully decoupled from which client or mode is rendering. The nice journal default (one block per entry) survives as the *default value* of that setting for journal-type posts — a default, not a hardwire, always overridable. You keep the convenience without welding it to the UI.

And **no nesting** is locked: a multi-paragraph block's variants are always whole-span alternatives for the entire block; no paragraph inside a block carries its own sub-fan. That's the rule that keeps "one block blooms at a time" coherent.

That genuinely closes the split design. The one inverse we haven't defined, and we'll need it for the spec: **merge.** When you combine two adjacent blocks back into one — say you Extracted a suffix and later regret it — what happens to their fans? The clean default that mirrors Divide-W: merging B and C mints a *fresh* block whose active variant is "active-of-B + active-of-C," with derived-from edges to both, leaving B and C intact as lineage rather than trying to cartesian-product their fans together. Does that match your intuition, or do you picture merge differently — and should I fold this whole split/extract/merge taxonomy into the requirements list as its own section before we move to the spec?

—

yeah i think that's all fine, though i also think you should be able to fan certain

things out and they shouldn't close on you unless you want them to. there can be a close all fans button if you want. and yeah for merge that's fine, the mirroring divide idea. I think that this is all pretty clear, the UI will definitely need reworking and thinking but this is feeling pretty solid and flexible thus far. the hardest part of this project is deciding what happens with blocks and variants in splits, merges, extracts, divides, etc. as well as the underlying structure and I think that we've reached somewhere pretty solid. make another list with all of our technical decisions that we've decided - nothing as precise as what's will be in the data table, but the decisions.

— claude —

Here's everything we've actually decided — the architectural commitments, not the schema rows.

Platform & stack

- Native Apple, not cross-platform — the text experience and your shortcut/capability needs justify it; the user is you (and people who think like you)
- macOS-first, targeting current OS (you control your own machine, so no backward-compat tax)
- SQLite via GRDB as the store; Markdown *export* so it's never a black box
- SwiftUI for chrome and all the non-editing surfaces; the editing core wraps a TextKit 2 NSTextView (single continuous buffer)
- Same model layer shared to iOS later, since mobile deliberately does *less*

Core philosophy

- The atomic unit is the **block**; everything points at blocks rather than owning them — non-destructive, pointer-based composition (same religion as ClipFarm)
- The whole system is a **directed graph of text blocks with typed edges**; every view is a traversal/query of that graph, not a bespoke feature
- **Rich substrate, lean surface**: the data model is always fully capable; any given UI exposes only a chosen subset and lets the rest stay latent
- Coarse, not forensic: the system preserves *choices and lineage*, never a word-by-word reconstruction of what changed

The stored objects (as concepts)

- **Block** — a real node with a permanent identity; holds nothing but identity + its variants
- **Variant** — one alternative text for a block's slot; the "fan" is the set of a block's variants; variants are opt-in and rare, not ambient
- **Commit** — an append-only savepoint *inside* a single variant (variant A can have commits 1, 2, 3); this is the block's version history
- **Edge** — a typed link between blocks (derived-from for lineage; version-of

for compare relationships, usable even between posts)

- **Post** — the entry / blog post / chapter; does **not contain** blocks, it's an *ordered list of pointers* to them
- **Reading** — a book/chapter; either hand-curated (manual ordered list) or smart (a tag/query); sections come from grouping rules, not container objects
- **Tag** — hierarchical (path-style), applied to posts and (cheaply) to blocks

Key model decisions

- **The chosen variant lives on the pointer, not the block** — so the same block can show version A in one reading and version B in another
- **Variants never span blocks** — a variant is always exactly one block's text; all cross-block relationships are edges, never variants
- **Internal newlines are content, not structure** — a newline edits a block's text; it is *not* a block split; blocks can span multiple paragraphs freely, and different variants can have different internal paragraph structure
- **No nesting** — a multi-paragraph block's variants are whole-span alternatives for the entire block; no paragraph inside a block carries its own sub-fan
- **Block size**: floor is the sentence, default is the paragraph, ceiling-less upward; boundaries snap to sentence/paragraph marks; default segmentation of fresh text is a **document setting** (decoupled from UI/mode), with journal posts defaulting to one-block-per-entry
- **Single ownership** — a block belongs to one post; no knowledge-base transclusion; dragging from the pool is **copy + an auto derived-from edge** (new ID, new owner, lineage preserved, no shared-mutation)
- **Posts are edges-only** — no post-level variant *resolution*; "two articles side by side" and sticky-note boards are spatial **UI** over read-only post renders, joined by version-of edges

Mutation, commits, locking, deletion

- **One rule underneath it all**: mutation is local and ephemeral; permanence is explicit and append-only
- Each variant has one live **working buffer**: mutable, continuously autosaved, ordinary command-Z undo — this is the only mutable text in the system
- **Commit** is a deliberate shortcut that appends an immutable savepoint; commits are append-only and linear *per variant*; autosave never creates committed history
- **Lock is fully orthogonal to commit** — a reversible guard, not a snapshot; lock a variant (freeze its buffer) or a post (freeze pointer order + chosen variants + forbid new commits); always un-lockable; locking a post auto-commits its live buffers

- A post can be marked **complete** (the post-level lock); journals can be fixed-length (e.g. 200 posts) or declared done and locked
- Rejected variants can be **hard-deleted** or **archived** into the block's history view; deleting whole blocks is cheap, with an optional per-post "deleted blocks" tray
- Branching a new variant **inherits** the current buffer's text; wanting a blank/new block is a *different* gesture

Split taxonomy (the hard core, now settled)

- Two distinct operations, not one overloaded "split":
- **EXTRACT** — source survives: a boundary-adjacent prefix/suffix leaves the active variant to become a new adjacent block (with derived-from); the source keeps its identity and its **entire fan** untouched; zero ceremony, always available; this is also how a sub-paragraph (e.g. conclusion-sentence) fan is created without sub-paragraph blocks being a headache
- **DIVIDE** — source is consumed into ordered successors:
  - single-variant block → trivial cut into B + C (the everyday case, no ceremony)
  - multi-variant block → the **only** place per-variant marking exists: you place the cut into each variant you want carried; B's fan = first-halves, C's fan = second-halves (lets you pair first-half-of-1 with second-half-of-3); unmarked variants aren't cut
- A middle span cannot Extract (it would leave the slot discontiguous) — pulling a middle piece is by nature a Divide
- **W resolution** (identity follows the whole) is the chosen rule, and it now lives in exactly one cell — multi-variant Divide: original block A is preserved whole and referenceable, unmarked variants stay on A as lineage, B and C are minted fresh; the divide is reversible and nothing is destroyed
- **MERGE** mirrors Divide-W: combining adjacent blocks mints a fresh block whose active variant is "active-of-B + active-of-C," with derived-from edges back to both; B and C stay intact as lineage rather than cartesian-producting their fans

Views (v1 is deliberately multi-view to stress-test the model)

- Multiple views is the *point* — each is a different traversal that exercises a different edge of the schema (folder → tags/membership; book/scrub → ordering + post-as-page; timeline → commit/variant timestamps; outline → derived-from); a view that's tortured to build reveals a weak spot in the model
- **Calm default, depth on demand**: at rest the document is seamless prose; block boundaries are invisible until a block is focused/hovered
- A small **corner badge appears only on blocks that have a fan** — a real

signal, not clutter; one-variant blocks show nothing
- **Fans stay open once expanded** and don't auto-close; there's a **close-all-fans** button; an "expand all fans" view renders every fan stacked/columned for read-down-and-prune
- Only one block is "open"/active at a time in the normal editor; everything unfocused renders as finished text
- Planned views: continuous **book view** (scroll through notes, each note scrolls internally), **book-scrub** (bottom scrubber over posts, page = a post, 50 entries = 50 pages regardless of length), **timeline**, **side-by-side** (text | v1 | v2 | text), **outline-left/draft-right** with optional tracking and derived-from connector mapping, **diff** with optional whitespace normalization, folder view, **sticky-note/canvas board** for posts
- Build order: prove the core (paragraph/sentence blocks in a calm editor + opt-in fans + readings driving book view) first; defer outline→draft connectors, the fancy scrubber, and whitespace-diff

Sorting

- Two timestamps per post: lastTouched (any keystroke) and a major-revision stamp that only updates when cumulative change crosses a threshold (or a new variant is committed) — so a typo never reshuffles everything
- A reading's sort spec picks which stamp to use; journals sort by entry date instead

Mobile

- Same store and model, smaller *surface* (not a simpler model)
- Mobile can create journal entries and **edit existing blocks' active text**, but exposes **no variant/fan gestures**
- Journal restrictions are a **UI profile**, never a separate data model — so a journal entry can later "graduate" into a full reworkable post with zero migration
- Sync (likely CloudKit, given iCloud+) is deferred to its own phase; the append-only-leaning mobile surface keeps conflicts tractable

That's the full set of commitments. The natural next step is turning this into the actual data-model spec — concrete tables, fields, edge types, and the exact state each operation writes — but only when you're ready to move from decisions to schema.